

Supervised Learning for Online Shoppers Purchasing Intention Prediction

Lukáš Kotek
luka26@ru.is

Luca Porfiri
luca26@ru.is

Sofia Cuccù
sofiacuc26@ru.is

September 21, 2026

Abstract

Understanding whether an online shopping session will ultimately lead to a purchase is a challenging binary classification problem, as most browsing sessions do not result in a transaction. In this project, we investigate this problem using the UCI Online Shoppers Purchasing Intention dataset [1], which contains 12,330 sessions described by 17 features and a binary target. The dataset is heavily imbalanced, with only 15.5% of sessions resulting in a purchase.

We develop a complete supervised learning pipeline consisting of a stratified 80/20 train-test split, exploratory data analysis, preprocessing with one-hot encoding and min-max scaling, and hyperparameter tuning through 5-fold cross-validation. We compare six classifiers—Logistic Regression, Decision Tree, Random Forest, Histogram-based Gradient Boosting, Support Vector Classifier with RBF kernel, and k-Nearest Neighbors—under two class-imbalance settings: no reweighting (*raw*) and balanced class weights. Among the evaluated models, Histogram-based Gradient Boosting achieves the best performance, with a macro-F1 of 0.801 in the raw setting (0.789 in the balanced setting) and an ROC-AUC of 0.932 on the held-out test set. This substantially improves upon the trivial majority-class baseline, which achieves 84.5% accuracy but an F1-score of 0. We further examine the effect of class reweighting across model families, compare our results with those reported in the original dataset paper [1], and conduct an additional experiment using SMOTE [2] on the same dataset.

1 Introduction

1.1 Problem Statement

Predicting the purchasing intention of online shoppers is a critical task for e-commerce platforms. Accurate predictions enable businesses to personalize marketing, optimize website design, improve purchasing and reduce cart abandonment. In this project we address a binary classification problem: given a set of features describing a user’s browsing session, predict whether the session will end with a purchase (**Revenue = True**) or not (**Revenue = False**).

1.2 Dataset

We use the **Online Shoppers Purchasing Intention Dataset** from the UCI Machine Learning Repository [3]. The dataset contains 12,330 sessions collected over a one-year period [1]. Each session is described by 17 features, including numerical variables (e.g., **PageValues**, **BounceRates**, **ProductRelated_Duration**) and categorical variables (e.g., **Month**, **VisitorType**). The target variable, **Revenue**, indicates whether the session ended with a purchase.

The dataset is severely imbalanced: 10,422 sessions (84.5%) did not result in a purchase, while only 1,908 sessions (15.5%) did. This imbalance is a key challenge throughout the project. No missing values are present in the dataset, and no additional imputation was required. During preprocessing, **BounceRates** was removed because it is almost perfectly correlated with **ExitRates** (Pearson $r = 0.91$). When two features carry essentially the same information, keeping both introduces multicollinearity without adding predictive power. We retain **ExitRates** as the representative of the pair. The models are therefore trained on 16 features.

1.3 Related Work

This dataset has been used in several follow-up studies, like Agustyaningrum et al. [4], that compared conventional machine learning classifiers with a deep neural network on the same data, reporting that a six-hidden-layer DNN reached an F1-score of 0.95 under their setup. Ajibade et al. [5] applied Logistic Regression, Random Forest, and XGBoost to the same dataset and reported Random Forest as the best of the three. Ajibade et al. [6] analysed the same dataset with a broader set of classical models (Logistic Regression, Random Forest, Decision Tree, K-Means, Linear Regression) and reported Random Forest as the best classifier with 89.2% accuracy. Their feature analysis identified **PageValues** and **ExitRates** as the most predictive features, which is consistent with what we observe in our own permutation importance results. In this project we focus on classical tree-based and linear classifiers, with explicit hyperparameter tuning and a controlled comparison of two class-imbalance strategies (no reweighting vs. **class_weight='balanced'**). We do not use deep learning, as our goal is to evaluate how far classical methods can go on this dataset with a carefully designed pipeline.

1.4 Benchmark

We define a trivial benchmark: a classifier that always predicts the majority class (**Revenue = False**). This benchmark achieves 84.5% accuracy but zero Recall on the positive class, which shows that accuracy alone is a poor metric for this imbalanced problem. Any meaningful model must outperform this benchmark on metrics such as F1-score, Recall, and AUC.

2 Process

2.1 Exploratory Data Analysis

Before designing the pipeline, we performed an exploratory analysis on the training data.

Correlation analysis. We computed the Pearson correlation matrix on the numerical features. Fig. 1 displays the correlation matrix, highlighting the strongest correlations.

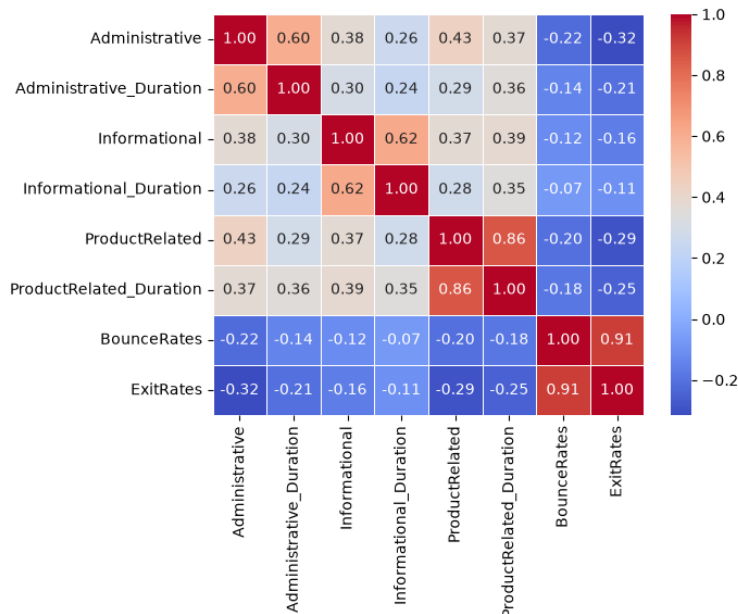


Figure 1: Correlation matrix of the dataset features. It shows a high correlation between `BounceRates` and `ExitRates`, and between `ProductRelated` and `ProductRelated_Duration`.

Two pairs showed a strong correlation:

- `BounceRates` and `ExitRates`: $r = 0.91$;
- `ProductRelated` and `ProductRelated_Duration`: $r = 0.86$.

Since `BounceRates` and `ExitRates` are almost redundant (both are Google Analytics metrics describing the fraction of sessions ending at a given page), we removed `BounceRates` to reduce multicollinearity. We kept both `ProductRelated` and `ProductRelated_Duration` because they capture different information: respectively a count and a time spent.

2.2 Data Splitting

We split the data into 80% for training (9,864 sessions) and 20% for testing (2,466 sessions), using `stratify=y` to preserve the class distribution in both sets. The test set is set aside and used only at the end to evaluate the final models. The training set, on the other hand, is used for cross-validation, allowing us to optimize hyperparameters to obtain the best model.

2.3 Outlier Analysis

The duration features (`Administrative_Duration`, `Informational_Duration`, `ProductRelated_Duration`) are heavily right-skewed, with maximum values far above their third quartile.

For example, on the training set `ProductRelated_Duration` has $Q_1 = 182.57$, $Q_3 = 1474.51$, and maximum 63,973.52. Fig. 2 shows the boxplots for these features.

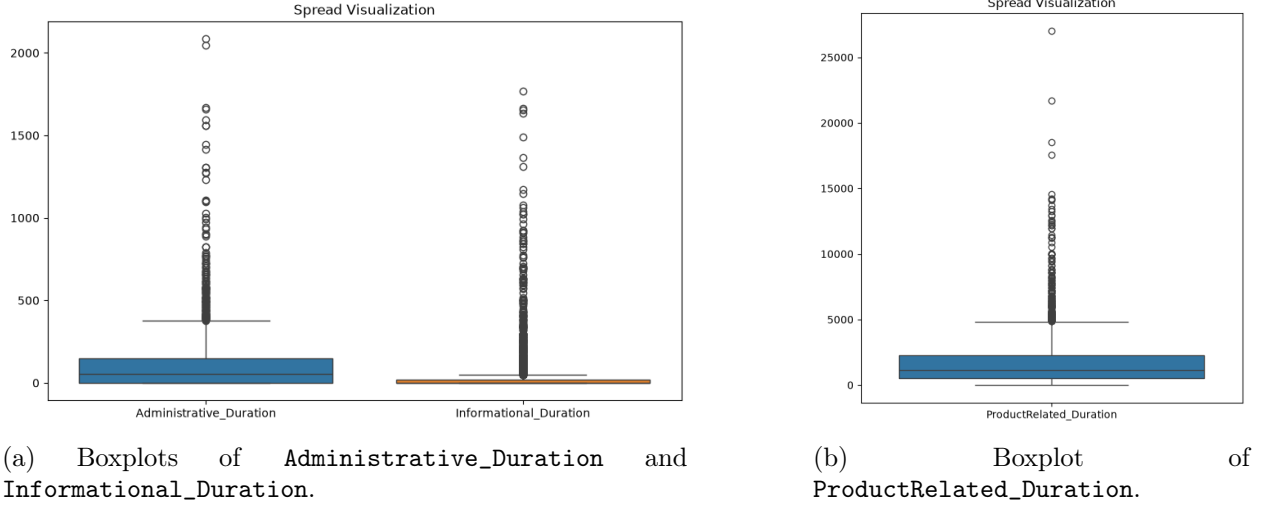


Figure 2: Boxplots of the three duration features on the training set. Extreme values are visible above the whiskers.

To assess whether these points should be removed, we used the **Interquartile Range (IQR)** method with a $2.5 \cdot \text{IQR}$ threshold, which is more conservative than the standard $1.5 \cdot \text{IQR}$ rule:

$$\text{Lower Bound} = \max(Q_1 - 2.5 \cdot \text{IQR}, 0) \quad (1)$$

$$\text{Upper Bound} = Q_3 + 2.5 \cdot \text{IQR} \quad (2)$$

Applying this rule to the training set gives an upper bound of 325.5 for `Administrative_Duration` and 4704.36 for `ProductRelated_Duration`. Under this rule, 5.59% of the training rows would be flagged for `Administrative_Duration` and 4.05% for `ProductRelated_Duration`. A closer inspection shows that the flagged sessions have a purchase rate of **26.50%** (`Administrative_Duration`) and **34.34%** (`ProductRelated_Duration`), compared to an overall training-set purchase rate of **15.47%**. In other words, the “outliers” are *more* likely to convert, not less: removing them would discard informative examples that carry strong signal about purchase intent.

For `Informational_Duration`, both the first and third quartiles are 0, so the IQR is 0 and the rule would identify any non-zero value as an outlier. This confirms that blindly applying the IQR rule to these features would be inappropriate.

We therefore decided not to remove any row from the dataset. The IQR bounds were computed only for analysis and are not used in the pipeline.

2.4 Preprocessing

After removing `BounceRates`, we identify two groups of features:

- **Numerical features:** `PageValues`, `ExitRates`, `Administrative`, `Administrative_Duration`, `Informational`, `Informational_Duration`, `ProductRelated`, `ProductRelated_Duration`, `SpecialDay`.

- **Categorical features:** `Month`, `VisitorType`, `OperatingSystems`, `Browser`, `Region`, `TrafficType`, `Weekend`. Note that `OperatingSystems`, `Browser`, `Region`, and `TrafficType` are stored as integers but are categorical identifiers.

We use a `ColumnTransformer` with:

- `MinMaxScaler` for numerical features (scales values to $[0, 1]$, suitable for distance-based models such as SVC and KNN). Scaling ensures that features with large ranges (e.g., `ProductRelated_Duration`) do not dominate the distance computation.
- `OneHotEncoder` for categorical features. This converts categorical identifiers into binary columns that the models can process, at the cost of increasing the dimensionality of the feature space.

After transformation, the feature space expands to **73 dimensions**.

The preprocessor was trained exclusively on the training set and subsequently applied to the test set. This avoids information leakage (data leakage).

2.5 Handling Class Imbalance

Given the severe imbalance (84.5% / 15.5%), we compare two strategies:

1. *raw*: train each classifier on the original, imbalanced training set;
2. *balanced*: train each classifier with `class_weight='balanced'`, which reweights the loss so that both classes contribute equally.

Note that `class_weight='balanced'` does not modify the dataset; it only changes how errors are weighted during training. This is different from oversampling techniques such as SMOTE [2], which generate synthetic minority samples and change the effective size of the training set. We chose `class_weight='balanced'` for the main comparison because it leaves the data distribution untouched and is therefore simpler to analyze it. We report an additional SMOTE experiment for completeness in Section 3.8. The full implementation of both the main pipeline and the SMOTE experiment is available in our public repository [7].

KNN does not accept a `class_weight` argument, so its results are identical in the two settings.

2.6 Model Selection

We select six models with different inductive biases:

- **Logistic Regression**: interpretable linear baseline.
- **Decision Tree**: single-tree model, useful to measure the benefit of ensembling.
- **Random Forest**: ensemble of decision trees, captures non-linear relationships.
- **Histogram-based Gradient Boosting (HistGB)**: state-of-the-art boosted ensemble, faster than classic Gradient Boosting on tabular data.
- **Support Vector Classifier (SVC)** with RBF kernel: effective in high-dimensional spaces.
- **k-Nearest Neighbors (KNN)**: distance-based baseline, included to measure how a non-parametric method performs after one-hot encoding.

2.7 Hyperparameter Tuning

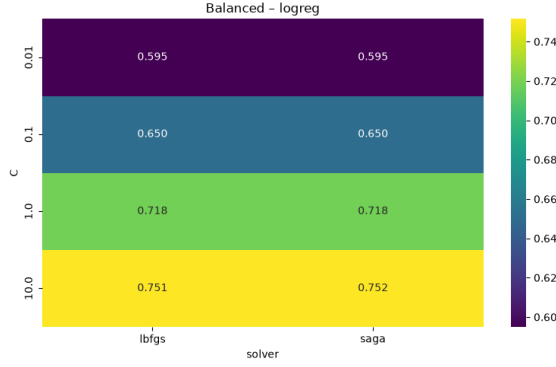
We use `GridSearchCV` with 5-fold `StratifiedKfold` cross-validation (with shuffling and a fixed random seed). The tuning metric is the **macro F1-score** (sometimes referred to as F1), which weights both classes equally and therefore does not reward trivial predictions of the majority class. The search spaces are reported in Table 1.

Table 1: Hyperparameter search spaces used in `GridSearchCV`.

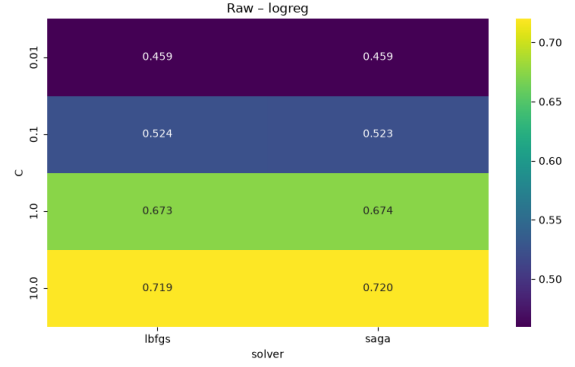
Model	Hyperparameter Grid
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$ $\text{solver} \in \{\text{lbfgs}, \text{saga}\}$
Decision Tree	$\text{max_depth} \in \{5, 10, 20\}$ $\text{min_samples_split} \in \{5, 10\}$ $\text{min_samples_leaf} \in \{1, 2, 4\}$
Random Forest	$\text{n_estimators} \in \{50, 100, 200\}$ $\text{max_depth} \in \{2, 5, 10\}$ $\text{min_samples_split} \in \{5, 10\}$ $\text{min_samples_leaf} \in \{1, 2, 4\}$
HistGradientBoosting	$\text{learning_rate} \in \{0.01, 0.1, 0.2\}$ $\text{max_iter} \in \{100, 200, 500\}$ $\text{max_leaf_nodes} \in \{15, 31, 63\}$
SVC (RBF)	$C \in \{0.1, 100, 1000\}$ $\text{gamma} \in \{0.001, 0.01, \text{auto}\}$
k-NN	$\text{n_neighbors} \in \{3, 9, 15, 33\}$ $\text{weights} \in \{\text{uniform}, \text{distance}\}$

The tuning results for each model in the two settings are visualized below. Fig. 7 shows the macro-F1 validation for KNN as a function of k and the weighting scheme. Fig. 4 shows the equivalent heatmap for the SVC as a function of C and γ . For models with more than two hyperparameters (Logistic Regression, Decision Tree, Random Forest, HistGB), we report the top-10 configurations sorted by validation macro-F1 in Fig. 3, Fig. 5, and Fig. 6.

Heatmaps and configuration plots help us understand the importance of certain hyperparameters for the model. The macro-F1 score of Logistic Regression is affected the most by the parameter C . It is the same for SVC. However, for lower C values, the importance of gamma increases. The most influential parameter of Random Forest is the **max_depth**, which is illustrated in Fig. 8. Additionally, Fig. 5 indicates that Random Forest tends to overfit more for the raw setting than the balanced ones. The opposite behavior is shown in Fig. 6 by HistGB. In terms of parameters, there is no parameter that has more influence than the others according to Fig. 6. Lastly, the KNN classifier is mostly affected by the value of K as expected.

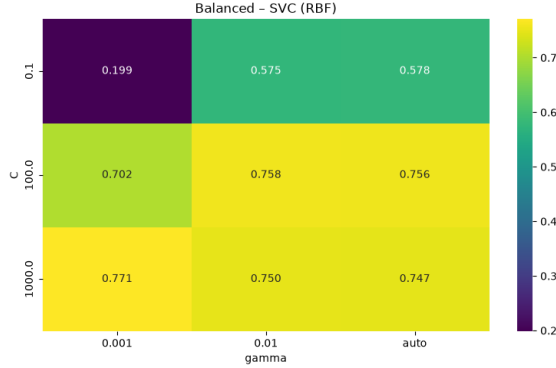


(a) Configuration heatmap of validation macro-F1 for Logistic Regression with `class_weight='balanced'`.

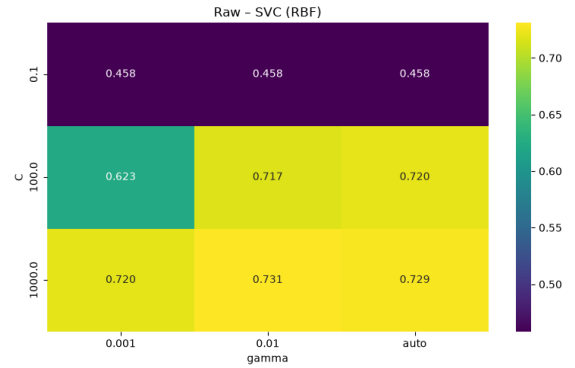


(b) Configuration heatmap of validation macro-F1 for Logistic Regression without class reweighting.

Figure 3: Configuration heatmap of validation macro-F1 configurations for Logistic Regression in the two settings.

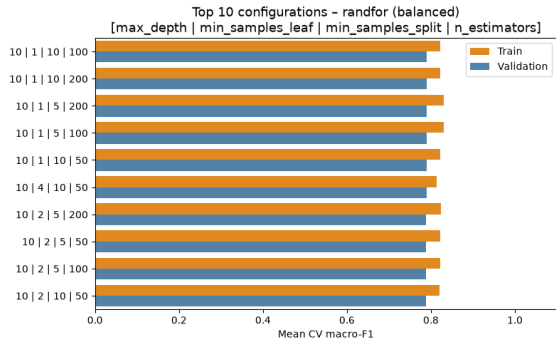


(a) Configuration heatmap of validation macro-F1 for the SVC (setting *balanced*).

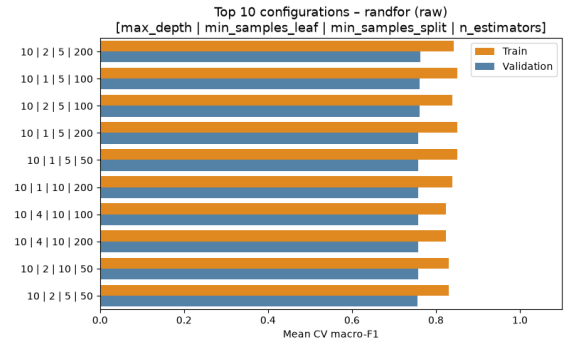


(b) Configuration heatmap of validation macro-F1 for the SVC (setting *raw*).

Figure 4: Configuration heatmap of validation macro-F1 for the SVC as a function of C and γ in the two settings.

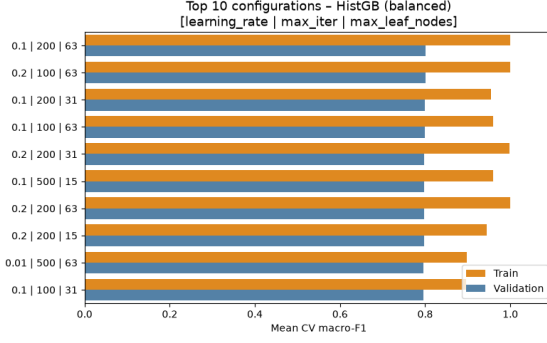


(a) Top-10 configurations for Random Forest with `class_weight='balanced'`.

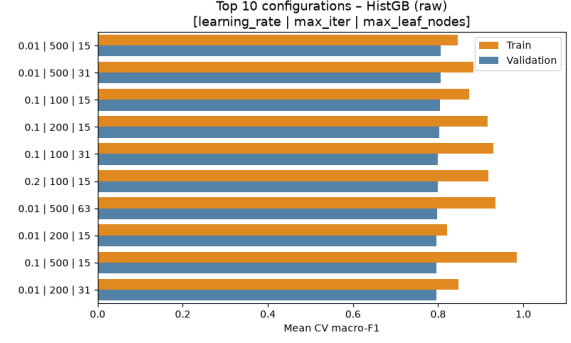


(b) Top-10 configurations for Random Forest without class reweighting.

Figure 5: Top-10 hyperparameter configurations for Random Forest in the two settings.



(a) Top-10 configurations for HistGB with `class_weight='balanced'`.



(b) Top-10 configurations for HistGB without class reweighting.

Figure 6: Top-10 hyperparameter configurations for HistGB in the two settings.

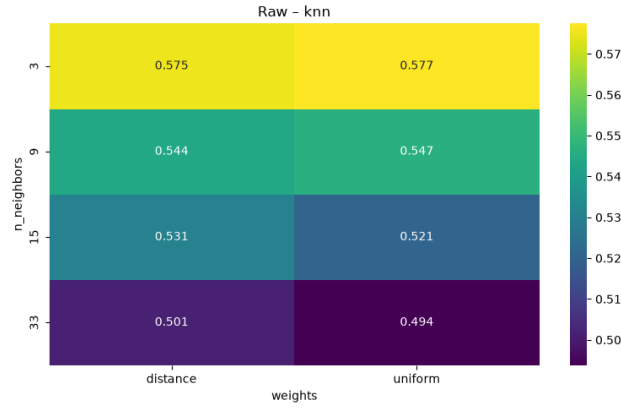


Figure 7: Configuration heatmap of validation macro-F1 for KNN as a function of k and weighting scheme. Since KNN has no `class_weight` argument, the results are identical in the *raw* and *balanced* settings.

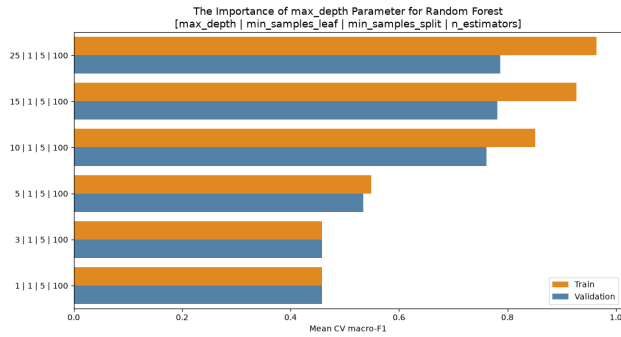


Figure 8: An illustration of how `max_depth` affects the train and validation score of Random Forest. Combined with Fig. 5, it shows that `max_depth` is the most influential parameter. Very large values lead to overfitting and very small values lead to underfitting, so the optimal value is one with slightly lower validation score but a smaller overfitting gap.

2.8 Evaluation Metrics

Due to class imbalance, accuracy alone is misleading. We report:

- **Precision:** When the model predicts a purchase, how often is it correct?
- **Recall:** fraction of real purchases that the model identifies.
- **F1 score on the positive class:** harmonic mean of Precision and Recall. This is the primary metric used to compare the models in the test set.
- **ROC-AUC:** threshold-independent measure of ranking ability [8]. ROC curves are insensitive to class distribution [8].
- **Precision-Recall curves:** as argued by Saito and Rehmsmeier [9], the precision-recall plot is more informative than the ROC plot when the positive class is rare.

3 Results

This section presents the experimental results in two settings: *raw* (no class reweighting) and *balanced* (`class_weight='balanced'`). We first report tuning results, then final performance on the held-out test set, and finally discuss overfitting, the effect of class weighting, and feature importance. An additional experiment with SMOTE is reported in Section 3.8.

3.1 Tuning Results

Table 2 reports the best hyperparameter configuration per model and setting, together with the corresponding cross-validation macro-F1.

Table 2: Best hyperparameters and cross-validation macro-F1 after tuning, for both settings.

Setting	Model	Best Hyperparameters	CV macro-F1
raw	HistGB	lr=0.01, max_iter=500, max_leaf_nodes=15	0.806
raw	tree	max_depth=5, min_samples_leaf=1, min_samples_split=5	0.801
raw	randfor	n=200, depth=10, min_leaf=2, min_split=5	0.762
raw	svc	$C = 1000$, $\gamma = 0.01$, kernel=rbf	0.731
raw	logreg	$C = 10$, solver=saga	0.720
raw	knn	$k = 3$, weights=uniform	0.577
balanced	HistGB	lr=0.1, max_iter=200, max_leaf_nodes=63	0.800
balanced	randfor	n=100, depth=10, min_leaf=1, min_split=10	0.790
balanced	svc	$C = 1000$, $\gamma = 0.001$, kernel=rbf	0.771
balanced	tree	max_depth=5, min_samples_leaf=1, min_samples_split=5	0.768
balanced	logreg	$C = 10$, solver=saga	0.752
balanced	knn	$k = 3$, weights=uniform	0.577

The full tuning process took approximately 19 minutes in the *raw* setting and 24 minutes in the *balanced* setting, for a total of about 43 minutes on the Colab CPU environment.

The validation macro-F1 score is not that much affected by using balanced weights. The largest difference across all models is 0.04 (SVC), and the Decision Tree score is actually lower with balanced

weights. The worst classifier is KNN, which may be due to the dataset’s feature space being too sparse. When comparing Decision Tree and Random Forest, the Decision Tree achieves a better validation score than the Random Forest, despite being a simpler model.

3.2 Final Evaluation on the Test Set

Table 3 shows the final performance on the held-out test set (2,466 sessions, never used during tuning). The primary metric is the macro-F1 score.

Table 3: Final model performance on the held-out test set. Best macro-F1 and AUC per setting are in bold.

Setting	Model	Accuracy	Precision	Recall	F1 (macro)	AUC	CV F1	Train F1
raw	HistGB	0.902	0.718	0.607	0.801	0.932	0.806	0.846
raw	tree	0.891	0.656	0.623	0.787	0.915	0.801	0.826
raw	randfor	0.895	0.821	0.408	0.743	0.916	0.762	0.843
raw	svc	0.883	0.740	0.380	0.718	0.856	0.731	0.776
raw	logreg	0.879	0.734	0.340	0.698	0.878	0.720	0.722
raw	knn	0.828	0.392	0.194	0.581	0.649	0.577	0.753
balanced	HistGB	0.885	0.616	0.681	0.789	0.915	0.800	0.999
balanced	randfor	0.857	0.524	0.825	0.776	0.915	0.790	0.822
balanced	svc	0.861	0.538	0.725	0.766	0.895	0.771	0.785
balanced	tree	0.835	0.481	0.851	0.755	0.906	0.768	0.787
balanced	logreg	0.833	0.474	0.736	0.736	0.888	0.752	0.761
benchmark	Always-False	0.845	0.000	0.000	0.000	0.500	–	–

The best overall model in each setting is HistGB, with $F1 = 0.801$ in *raw* and $F1 = 0.789$ in *balanced*. The two configurations are close on F1, but the *raw* model reaches a higher AUC (0.932 vs. 0.915). Random Forest is the runner-up in both settings and is the only model with a precision above 0.80 in the *raw* setting. Interestingly, Random Forest achieves high precision in the *raw* setting and high recall in the *balanced* setting. This is observable across most of the models. Therefore, one setting would be preferable to the other depending on which metric is valued more.

3.3 Confusion Matrices

Fig. 9 shows the confusion matrices of all six classifiers on the test set, in both settings. Rows correspond to the true labels and columns to the predicted labels.

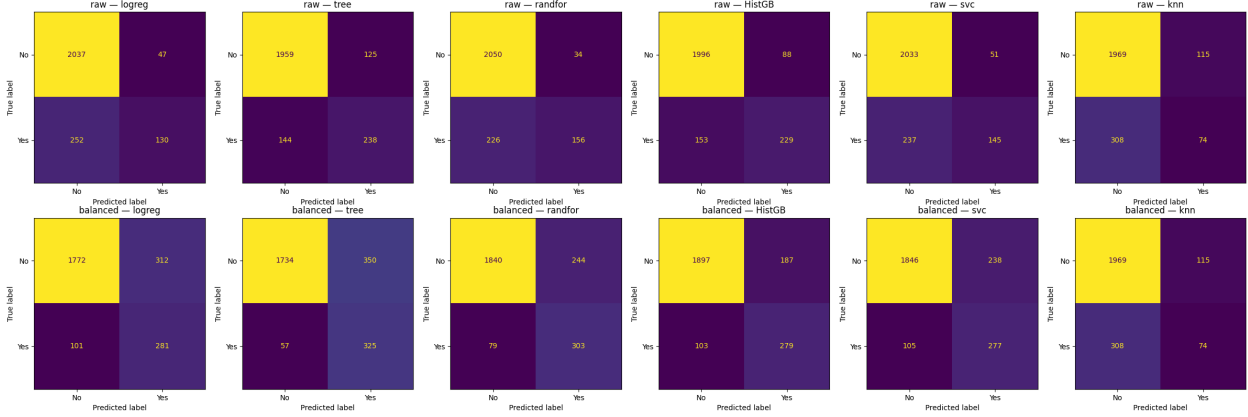


Figure 9: Confusion matrices on the test set for all six classifiers, in the *raw* (top row) and *balanced* (bottom row) settings.

The matrices confirm the numeric picture from Table 3. In the *balanced* setting, all models reduce the number of false negatives at the cost of more false positives, which is the visible effect of `class_weight='balanced'` on the decision boundary.

KNN is the only model whose confusion matrix is identical in the two settings, as expected: it does not accept a `class_weight` argument. Its pattern is also the most unbalanced of the six models, with many false negatives and very few true positives.

3.4 Overfitting Analysis

Because both the cross-validation score (Table 2) and the test score (Table 3) are macro-F1, the two are directly comparable. The gap between them is therefore a meaningful measure of generalization from the validation folds to the held-out test set.

In the *raw* setting, the CV–test gaps are small across the board (at most 0.02). HistGB, for example, drops only from 0.806 (CV) to 0.801 (test). Similar behavior is observed for Decision Tree and SVC.

In the *balanced* setting, the CV–test gaps remain modest (0.011 for HistGB, 0.014 for Random Forest). However, the training scores indicate that HistGB achieves a training macro-F1 of 0.999, while its CV score is only 0.800. This large train–CV gap indicates that the model substantially overfits the training folds, yet the overfitting is sufficiently stable that the subsequent drop from CV to test remains small.

Overall, no model exhibits a large degradation from cross-validation to the test set. The performance differences between the *raw* and *balanced* settings are therefore driven mainly by the effect of class reweighting on the decision boundary rather than by unstable overfitting.

3.5 Effect of Class Weighting

Comparing the two settings reveals a clear pattern:

- **Logistic Regression** gains 0.038 in macro-F1 (0.698 $\rightarrow$ 0.736);
- **SVC** gains 0.048 (0.718 $\rightarrow$ 0.766);
- **Random Forest** gains 0.033 (0.743 $\rightarrow$ 0.776);
- **HistGB** slightly decreases (0.801 $\rightarrow$ 0.789);

- **Decision Tree** also decreases ($0.787 \rightarrow 0.755$);

The pattern is consistent with the intuition that class reweighting helps models whose loss is directly sensitive to class frequencies, and has little effect on models that already partition the feature space using thresholds. The improvement is largest for the linear and kernel methods, moderate for bagged trees, and slightly negative for the tree-based models.

3.6 ROC and Precision-Recall Curves

Fig. 10 shows the ROC and precision-recall curves for HistGB and Random Forest in the *balanced* setting, the two best models on the test set.

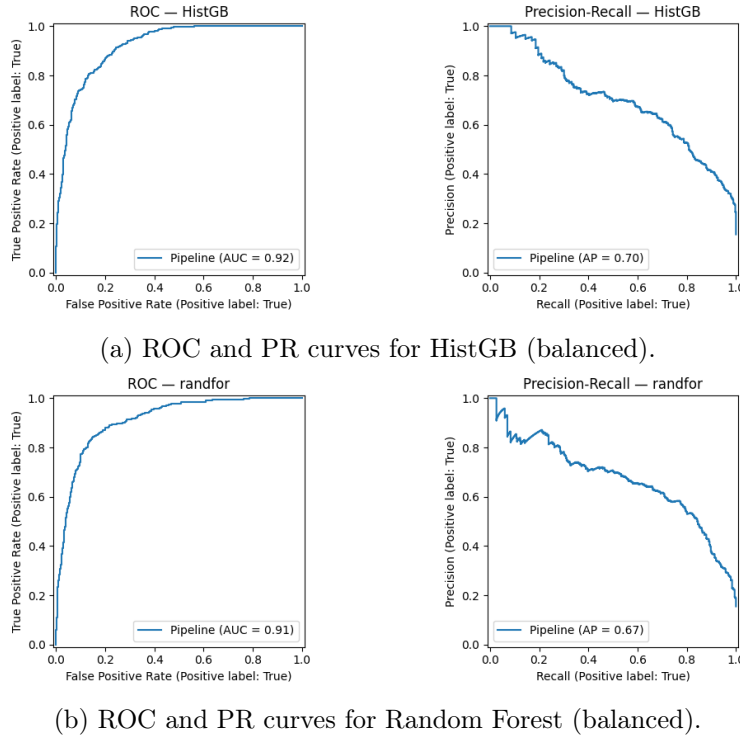


Figure 10: ROC and precision-recall curves for the two best models. HistGB reaches $AUC = 0.92$ (ROC) and $AP = 0.70$ (PR).

The ROC curve of HistGB remains well above the diagonal, with an AUC of 0.92, confirming good ranking performance. The precision-recall curve, which is more informative for imbalanced data [9], shows average precision (AP) of 0.70 for HistGB. This is the value that matters most in practice, since the positive class is only 15.5% of the test set.

3.7 Feature Importance

Fig. 11 shows the permutation importance (scoring = F1) computed on the test set for the two best models in the *balanced* setting.

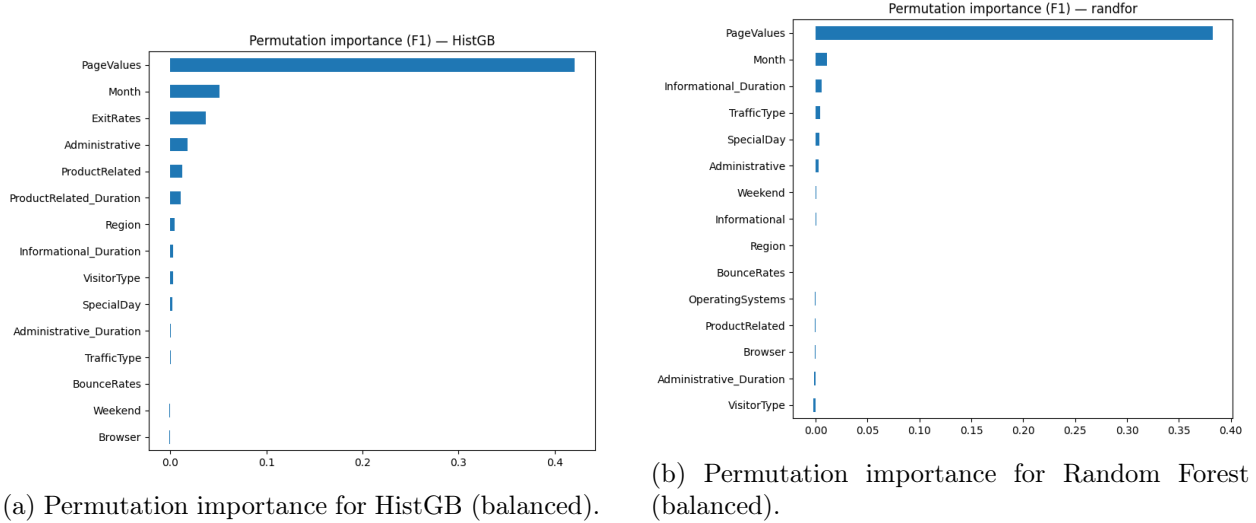


Figure 11: Permutation importance on the test set for HistGB and Random Forest. **PageValues** dominates in both models.

For HistGB, the most important features are **PageValues** (by a large margin), followed by **Month**, **ExitRates**, and **Administrative**. For Random Forest, **PageValues** still dominates, but the importance of the remaining features is much more evenly distributed. This is consistent with the general observation that boosted trees concentrate their capacity on a small number of high-signal features, while bagged trees spread it across many.

PageValues, being the most important feature (as in [1]), makes all the trained models hard to use in real-world applications. Page value is a metric that can help you understand how valuable each page on your site is for generating revenue and leads. However, it is computed retrospectively, making it impossible to use for real-time prediction. This would lead to much worse predictions of most of the presented models.

3.8 Additional Experiment: SMOTE

As an additional experiment, we tested SMOTE [2] as an alternative class-imbalance strategy on the same dataset. SMOTE generates synthetic samples for the minority class by interpolating between existing minority points and their k nearest minority neighbours. In this experiment, SMOTE was applied within the `imblearn` pipeline, so that synthetic samples were generated only from the training folds of each cross-validation split, thereby avoiding data leakage. The experiment used a slightly different preprocessing and hyperparameter grid than the main comparison; the exact procedure is described in the original code and is not repeated here.

Table 4: Test set performance of SMOTE-based models. These numbers come from a separate experiment with a different pipeline and are indicative only.

Model	Accuracy	Precision	Recall	F1	AUC
HistGB	0.886	0.612	0.730	0.798	0.929
Random Forest	0.882	0.596	0.730	0.792	0.916
Logistic Regression	0.848	0.506	0.736	0.753	0.889
SVC	0.852	0.522	0.550	0.724	0.823

On this dataset, SMOTE and class reweighting produce similar macro-F1 scores for HistGB (0.798 vs. 0.789) and Random Forest (0.792 vs. 0.776), suggesting that the choice between the two has a small effect. Class reweighting has the practical advantage of not modifying the data distribution, which makes the pipeline simpler to reason about. Because the SMOTE pipeline differs from the main pipeline in feature preprocessing and hyperparameter grid, the comparison should be read as indicative rather than strictly controlled.

The SMOTE pipeline is implemented in a separate notebook, available in the same repository as the main experiment [7].

3.9 Summary

Across both settings, HistGB is the best-performing model, with macro-F1 = 0.801 in *raw* and 0.789 in *balanced*, and AUC above 0.91. The two class-imbalance strategies produce similar macro-F1 for HistGB but affect the other models differently: `class_weight='balanced'` improves macro-F1 by roughly 0.03–0.05 for Logistic Regression, SVC, and Random Forest, slightly decreases it for HistGB and Decision Tree, and has no effect on KNN. The trivial majority-class benchmark reaches 0.845 accuracy but zero F1, which confirms that accuracy alone is misleading on this dataset.

4 Conclusions

4.1 Interpretation of Results

HistGB is the strongest model in both settings. In the *balanced* setting it reaches macro-F1 = 0.789 and AUC = 0.915; in the *raw* setting it reaches macro-F1 = 0.801 and AUC = 0.932. The raw model has a small edge on both metrics, and it is also the configuration where class reweighting is least helpful, as we discuss in Section 4.2.

Random Forest comes second in both settings, and in the raw setting it is the only model whose precision exceeds 0.80. The price of that precision is a low recall (0.408), so it catches less than half of the actual buyers. Logistic Regression sits at the opposite end: it has a much higher recall (0.736) but the lowest precision (0.474). In practice, the choice between these two comes down to which type of error is more expensive for the business.

All models clearly beat the trivial majority-class baseline. The benchmark reaches 84.5% accuracy simply by predicting “no purchase” every time, but its F1 is zero, which is why accuracy on its own tells us very little about this dataset. Even the best models, however, sit in the 0.5–0.6 range on precision: when they flag a session as a likely purchase, they are right roughly half the time. This is what happens when only 15.5% of the sessions are positive: pushing recall up requires accepting many false positives, and no classical model we tried can avoid this trade-off.

KNN is the weakest model, with a macro-F1 of 0.581 in both settings. This is not surprising given that one-hot encoding pushes the feature space to 73 dimensions.

4.2 Effect of Class Reweighting

A key finding of this project is that `class_weight='balanced'` does not affect all models equally. On macro-F1, it improves performance for Logistic Regression (+0.038), SVC (+0.048), and Random Forest (+0.033), while it slightly decreases for HistGB (-0.012) and Decision Tree (-0.032). KNN is unaffected because it does not support class reweighting.

This is consistent with the intuition that reweighting helps models whose loss is directly sensitive to class frequencies (linear and kernel methods) and has little effect on models that already partition

the feature space using thresholds (tree-based methods). The difference is small for HistGB because boosted trees are already robust to imbalance: the boosting procedure naturally focuses on hard-to-classify examples, which tend to be the minority class.

4.3 Overfitting

The most expressive models show a modest gap between cross-validation macro-F1 and test macro-F1. HistGB, for example, reaches 0.806 in cross-validation and 0.801 on the test set, a gap of only 0.005. Because both metrics are macro-F1, this gap is a direct measure of the difference between validation and test performance. Decision Tree and SVC show similar small gaps, which suggests that their decision boundaries generalize well.

4.4 Comparison with Literature

The original dataset paper by Sakar et al. [1] reports an accuracy of 87.24% and an F1 score of 0.86 with a Multi-Layer Perceptron on an oversampled version of the dataset, using a 70/30 train/test split and 100 random repetitions. Our HistGB achieves comparable accuracy (88.5% in the *balanced* setting), but their F1 is computed only on the positive class, whereas we report macro-F1 in Table 3; a direct numerical comparison is therefore not exact. Three differences in the experimental setup may explain the remaining gap: (i) Sakar et al. use a 70/30 split with a larger test set of 3,699 sessions; (ii) their best result is obtained after feature selection, keeping only the six most relevant features; (iii) they average over 100 repetitions, which reduces the variance of their estimate.

Ajibade et al. [5] report F1-scores around 93% on the same dataset using Random Forest and XGBoost combined with SMOTE. The exact preprocessing order and train-test split protocol are not fully specified in their paper, so their results are not directly comparable to ours. Their higher F1-scores are likely due in part to differences in the evaluation protocol; we therefore report them as a reference rather than a benchmark to match.

4.5 Hyperparameter Importance

The tuned configurations differ between the two settings, which is expected since the loss is reweighted in the balanced case. Some patterns are worth noting:

- **HistGB** prefers a small learning rate with many iterations in the raw setting (`lr=0.01`, `max_iter=500`, `max_leaf_nodes=15`), and a larger learning rate with fewer iterations in the balanced one (`lr=0.1`, `max_iter=200`, `max_leaf_nodes=63`).
- **Random Forest** selects `max_depth=10` in both settings. The main differences are the number of trees (200 in raw, 100 in balanced) and `min_samples_split` (5 in raw, 10 in balanced).
- **Logistic Regression** prefers weaker regularisation (`C=10`) with the `saga` solver in both settings.
- **SVC** requires a large `C` in both settings (`C=1000`), but the optimal `gamma` differs (0.01 in raw, 0.001 in balanced).

Tuning improved the macro-F1 on the validation folds for every model we tested, and did not change the ranking of the models: HistGB remained at the top in both settings. We do not report a numerical improvement over the untuned baselines because the baseline experiments used a different scoring metric, and the two are therefore not directly comparable.

5 Future Work

The current results leave several directions open.

- **Feature engineering and selection.** Derived features such as the average time per product page (`ProductRelated_Duration / ProductRelated`) could capture engagement more directly than either quantity alone. After one-hot encoding the space has 73 dimensions, many of which carry little signal; applying an mRMR filter, as in Sakar et al. [1], or PCA could improve the weaker models (KNN, SVC) and reduce training time.
- **Threshold optimisation and calibration.** All results use the default 0.5 threshold. In a real deployment, missing a likely buyer is usually more costly than showing a promotion to a non-buyer, so a lower threshold would likely improve the business-relevant trade-off. Calibrating HistGB’s scores with Platt scaling would also make its output directly interpretable as a purchase probability.
- **Alternative imbalance strategies.** We compared *raw* and *balanced* settings, and reported an additional SMOTE experiment. ADASYN, Borderline-SMOTE or SMOTE combined with Tomek links could be evaluated under the same protocol, always applying resampling strictly inside the cross-validation loop to avoid leakage.
- **Ensemble of heterogeneous models.** A soft-voting or stacking ensemble combining HistGB, Random Forest, and Logistic Regression could reduce prediction variance: the three base models capture different aspects of the data (boosted trees for the strongest signal, bagged trees for robustness, linear models for simple patterns).
- **Deep learning baselines.** Sakar et al. [1] reached $F1 = 0.86$ with an MLP and an LSTM after feature selection and oversampling. Reproducing their setup on the same folds would give a fair classical-vs-deep comparison on this dataset.

What we would do differently. Given the results, we would redesign the tuning protocol in two ways. First, we would repeat the 5-fold cross-validation with multiple seeds and report averages with confidence intervals, since a single run gives a noisy estimate, especially for KNN. Second, we would drop KNN from the model pool from the start: it is useful as a sanity check, but it consumes tuning time without producing a competitive candidate, and that time could be reinvested in a wider hyperparameter search for HistGB.

References

- [1] C. Okan Sakar, S. Olcay Polat, Mete Katircioglu, and Yomi Kastro. Real-time prediction of online shoppers’ purchasing intention using multilayer perceptron and lstm recurrent neural networks. *Neural Computing and Applications*, 31(10):6893–6908, 2019. doi: 10.1007/s00521-018-3523-0.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.
- [3] Kastro Sakar. Online shoppers purchasing intention dataset. <https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset>, 2018. Accessed on 16 September 2026.

- [4] Cucu Ika Agustyaningrum, Muhammad Haris, Riska Aryanti, and Titik Misriati. Online shopper intention analysis using conventional machine learning and deep neural network classification algorithm. *Jurnal Penelitian Pos dan Informatika*, 2021.
- [5] Samuel-Soma M. Ajibade, Ghassan Saleh ALDharhani, Olumide Simeon Ogunnusi, Muhammed Basheer Jasser, Anthonia Oluwatosin Adediran, Olamide Emmanuel Ayodele, V. E. Sathishkumar, Kayode A. Akintoye, and Ehineni Olufunke Janet. Machine learning classification of online shopper purchasing intentions. In *2025 International Conference on NexGen Networks and Cybernetics (IC2NC)*. IEEE, 2025. URL <https://ieeexplore.ieee.org/document/11376387>.
- [6] Samuel-Soma M. Ajibade, Anthonia Oluwatosin Adediran, Kayode A. Akintoye, Olumide Simeon Ogunnusi, Muhammed Basheer Jasser, and Olamide Emmanuel Ayodele. Behavioural insights into online shoppers’ purchase intention using machine learning models. *Journal of AI-Driven Communication Engineering*, 2025.
- [7] Lukáš Kotek, Luca Porfiri, and Sofia Cuccù. Online shoppers purchasing intention — supervised learning. <https://github.com/sofiacuccu00/online-shoppers-purchase-prediction.git>, 2026.
- [8] Tom Fawcett. An introduction to roc analysis, 2006.
- [9] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets, 2015.